

NISHCODEZZ — SOFTWARE ENGINEERING INTERVIEW PLAYBOOK

Stop preparing randomly. Start preparing with a system.

Getting a software engineering job is not only about solving LeetCode.

You need a **strong profile, the right DSA patterns, good projects, referrals, interview communication, system design, and offer negotiation.**

This playbook is my practical roadmap for going from:

“I am preparing for interviews” → “I am getting interviews” → “I am converting offers.”

01 — BUILD A PROFILE THAT GETS SHORTLISTED

Before spending 4 hours on DSA every day, make sure your profile is not losing opportunities before the interview even starts.

Resume

Your resume has one job:

Make the recruiter want to talk to you.

Keep it simple

- 1 page if you're a fresher/early-career candidate.
- Use a clean, ATS-readable format.
- Avoid unnecessary graphics, tables, skill bars and fancy designs.
- Use keywords that genuinely match the job description.
- Put your strongest projects and experience first.
- Quantify impact wherever possible.

Instead of:

Worked on backend optimisation.

Write:

Reduced API latency by 35% by introducing caching and query optimisation.

Recommended order

Header

Name | Email | Phone | LinkedIn | GitHub | LeetCode

Education

Technical Skills

Experience

Projects

Achievements / Certifications

Don't fill your resume with everything you've ever done.

Keep the strongest evidence.

02 — FIX YOUR LINKEDIN + GITHUB

LinkedIn

Your LinkedIn should answer three questions quickly:

Who are you? What can you build? What role are you looking for?

Do this:

- Use a professional profile picture.
- Write a specific headline.
- Mention your strongest technologies.
- Add projects and relevant experience.
- Keep your About section short.
- Turn on Open to Work if appropriate.
- Share useful technical content occasionally.

Example headline:

Software Engineer | Java + Spring Boot + React | DSA | Building Scalable Systems

Don't write:

Student at XYZ College.

Your headline should sell your **direction**, not just your current status.

GitHub

GitHub should work as your technical proof.

- Pin 3–5 strong repositories.
- Write proper READMEs.
- Explain architecture and setup.
- Add screenshots/demo links where useful.
- Keep commits meaningful.
- Build at least 2–3 projects that you can confidently explain in an interview.

A GitHub with 30 abandoned repositories is less useful than 3 projects you actually understand.

03 — DSA: LEARN PATTERNS, NOT ANSWERS

The biggest mistake

Opening LeetCode and solving random questions every day.

You might reach 300 solved problems and still struggle with a new interview question.

The target isn't:

“How many problems have I solved?”

The target is:

“How many problem patterns can I recognise?”

THE CORE DSA PATTERNS

Pattern	Where it appears
Two Pointers	Sorted arrays, pairs, opposite-end processing
Sliding Window	Subarrays and substrings
Hashing	Frequency, lookup, duplicates
Fast & Slow Pointers	Linked lists and cycle detection
Binary Search	Sorted data and answer-space search
Stack / Monotonic Stack	Next greater, brackets, histogram-style problems
BFS / DFS	Trees, graphs, grids
Heap / Priority Queue	Top-K, scheduling, streaming
Backtracking	Subsets, permutations, combinations
Greedy	Local decisions leading to global optimisation
Dynamic Programming	Overlapping subproblems and optimisation
Graph Algorithms	Shortest path, connectivity, dependencies
Union Find	Connected components and dynamic connectivity
Intervals	Overlapping ranges and scheduling

A better learning cycle

Pattern → Understand → Solve easy → Solve medium → Solve unfamiliar → Revise

That is much more valuable than blindly increasing your problem count.

04 — YOUR 12-WEEK DSA ROUTINE

Weeks	Focus	Target
1–2	Arrays + Strings + Hashing	25
3–4	Two Pointers + Sliding Window	20
5–6	Linked List + Stack + Queue	20
7–8	Trees + BFS + DFS	25
9–10	Graphs + Dynamic Programming	30
11	Heap + Backtracking + Tries	15
12	Company questions + Mock Interviews	15 + 5 mocks

Target: ~150 problems.

But don't chase the number.

If you understand 100 problems deeply, that's better than memorising 300 solutions.

HOW I WOULD PRACTISE EVERY PROBLEM

Step 1 — Think first

Spend 20–30 minutes trying yourself.

Step 2 — Take a hint

If you're stuck, understand the direction instead of immediately reading the full solution.

Step 3 — Code from scratch

Close the solution and implement it yourself.

Step 4 — Explain it

Say the approach out loud as if an interviewer is listening.

Step 5 — Revisit

Come back after:

3 days → 1 week → 1 month

Step 6 — Change the problem

Ask yourself:

“What happens if the constraint changes?”

That is where real interview-level understanding starts.

05 — DSA RESOURCES

NeetCode

<https://neetcode.io/roadmap>

Great for pattern-based preparation.

Striver A2Z

<https://takeuforward.org/strivers-a2z-dsa-course/strivers-a2z-dsa-course-sheet-2/>

Structured DSA from basics to advanced.

Striver SDE Sheet

<https://takeuforward.org/interviews/strivers-sde-sheet-top-coding-interview-problems/>

Focused interview preparation.

Blind 75

<https://leetcode.com/discuss/general-discussion/460599/blind-75-leetcode-questions>

Compact set of commonly discussed interview problems.

LeetCode Company Questions

<https://leetcode.com/company/>

Useful when targeting a particular company.

GeeksforGeeks

<https://www.geeksforgeeks.org/data-structures/>

Good for concepts and additional practice.

06 — DON'T IGNORE AI

Software engineering interviews are changing.

You don't need to become an AI researcher to benefit from AI.

For a modern software engineer, learn:

- LLM fundamentals
- Prompting
- Embeddings
- Vector databases
- RAG
- AI APIs
- Tool calling
- AI agents
- Evaluation
- Building AI-powered applications

My AI Learning Playlist

I have created a dedicated playlist for learning AI:

AI Learning Playlist:

<https://openup.to/islrygg/>

Use it alongside your normal DSA + development preparation.

The goal is not to add “AI” to your resume.

The goal is to actually build something with it.

07 — BUILD PROJECTS THAT ARE INTERVIEW-WORTHY

Don't build another:

- Calculator
- To-do app
- Basic weather app
- Simple CRUD app

Instead, build projects where you can discuss **architecture, scale, trade-offs and engineering decisions**.

Example project ideas

AI Resume Analyzer

- LLM
- RAG
- Vector database
- Authentication
- File processing

AI SQL Assistant

- Natural language → SQL
- Database integration
- Query validation
- Result explanation

Scalable Notification System

- Kafka
- Redis
- Workers
- Retry mechanism
- Monitoring

Job Tracker

- React

- Spring Boot/Node
- PostgreSQL
- Authentication
- Search/filtering
- Scheduled notifications

The project itself isn't what impresses the interviewer.

Your understanding of the project does.

08 — GET MORE INTERVIEWS

Don't depend only on “Apply”

Use three channels:

1. Direct Applications

Apply through:

- LinkedIn
- Naukri
- Instahyre
- Wellfound
- Company career pages

2. Referrals

Find employees at your target company.

A good order:

Alumni → Friends/known connections → Relevant employees → Recruiters

3. Direct Outreach

Keep your message short.

Example:

Hi [Name], I noticed you're working at [Company] as a [Role]. I'm currently targeting software engineering opportunities and have experience with [relevant skills]. I came across [Job ID/Role] and it looks closely aligned with my background. Would you be open to referring me? Happy to share my resume. Thanks!

Don't send the same paragraph to 100 people.

Personalise the first 1–2 lines.

09 — INTERVIEW PREPARATION

Round 1: Online Assessment

Practise both:

Problem solving + speed

Before the test:

- Understand the platform.
 - Practise timed sessions.
 - Read constraints carefully.
 - Keep track of time.
 - Don't spend 40 minutes stuck on one question.
-

ROUND 2 — CODING INTERVIEW

Use this structure:

1. Clarify

Understand the problem and constraints.

2. Brute Force

If useful, briefly establish the obvious solution.

3. Optimise

Move towards the best approach.

4. Code

Write clean, readable code.

5. Test

Walk through examples and edge cases.

6. Complexity

Always mention time + space complexity.

Don't do this:

- ✗ Start coding immediately
- ✗ Stay silent for 20 minutes
- ✗ Ignore edge cases
- ✗ Say “done” without testing
- ✗ Memorise code you can't explain

Interviewers are evaluating your **problem-solving process**, not just the final output.

10 — SYSTEM DESIGN

For SDE-2+ interviews, system design becomes increasingly important.

Start with these:

- URL Shortener
- Instagram
- Rate Limiter
- Notification System
- Chat Application
- File Storage System

Learn this sequence:

Requirements → Estimation → APIs → High-Level Design → Database → Scaling → Bottlenecks → Trade-offs

Free resource

<https://github.com/donnemartin/system-design-primer>

Don't memorise diagrams.

Understand **why each component exists**.

11 — BEHAVIOURAL / HR

Prepare stories using:

Situation → Task → Action → Result

Have examples ready for:

- A difficult technical problem
- A disagreement with a teammate
- A mistake/failure
- Taking ownership
- Working under pressure
- Learning something quickly
- A project you're proud of
- A time you received difficult feedback

Prepare these answers before the interview

Tell me about yourself

Why this company?

Why should we hire you?

Why are you switching?

What are your strengths/weaknesses?

What are your salary expectations?

Do you have any questions for us?

Never enter an interview without preparing your introduction.

12 — MOCK INTERVIEWS

You need to practise the **interview**, not just the questions.

Useful platforms

Pramp

<https://www.pramp.com/>

Peer-to-peer mocks.

Google Interview Warmup

<https://grow.google/certificates/interview-warmup/>

Interview practice.

LeetCode Interview

<https://leetcode.com/interview/>

Timed coding practice.

Interviewing.io

<https://interviewing.io/>

Practice with engineers.

13 — NEGOTIATE YOUR OFFER

Once you get an offer, don't immediately say yes.

Research:

- Base salary
- Bonus
- Stock/RSUs
- Joining bonus
- Location

- Remote/hybrid expectations
- Growth opportunities
- Role level

Salary research

Levels.fyi

<https://www.levels.fyi/>

AmbitionBox

<https://www.ambitionbox.com/>

Glassdoor

<https://www.glassdoor.com/>

If you have multiple offers, compare the **complete package**, not just CTC.

YOUR 90-DAY GAME PLAN

DAYS 1–15

Fix the foundation

- Resume
- LinkedIn
- GitHub
- Choose target companies
- Start DSA patterns

DAYS 16–45

Build problem-solving ability

- DSA daily
- Revise old questions
- Start one strong project
- Begin AI learning

DAYS 46–65

Start going aggressive

- Applications
- Referrals
- Networking
- Company-specific questions
- Continue DSA

DAYS 66–80

Interview mode

- Mock interviews
- OA practice
- Coding rounds
- System design
- Behavioural stories

DAYS 81–90

Close the gaps

Look at your actual interview feedback.

If you're failing:

- DSA → fix patterns
- Coding → improve communication/testing
- Resume → improve positioning
- System design → practise architecture
- HR → improve stories

Don't keep studying everything equally. Fix the bottleneck.

THE NISHCODEZZ INTERVIEW CHECKLIST

PROFILE

- Resume is clean and ATS-friendly
- LinkedIn clearly shows my target role
- GitHub has 3 strong projects
- Projects have measurable/technical impact

DSA

- I know the major patterns
- I can solve without looking at solutions
- I revise previously solved problems
- I can explain my approach aloud

DEVELOPMENT

- I have 2–3 projects I can explain deeply
- I understand my tech stack
- I know the architecture of my projects
- I can discuss trade-offs

AI

- I understand basic LLM concepts
- I have built at least one AI project
- I understand RAG/tool calling/agents at a practical level
- I have gone through the Nishcodezz AI playlist

APPLICATIONS

- I apply consistently
- I use referrals
- I network with relevant people
- I check company career pages

INTERVIEWS

- I clarify before coding
- I explain my approach
- I test my code
- I know complexity
- I have mock interview experience
- I have STAR stories ready

OFFER

- I know the market range
 - I compare total compensation
 - I understand the role/level
 - I negotiate professionally
-

QUICK RESOURCE SHEET

DSA

- NeetCode — <https://neetcode.io/roadmap>
- Striver A2Z — <https://takeuforward.org/strivers-a2z-dsa-course/strivers-a2z-dsa-course-sheet-2/>
- Striver SDE — <https://takeuforward.org/interviews/strivers-sde-sheet-top-coding-interview-problems/>
- LeetCode Company Questions — <https://leetcode.com/company/>

AI

- Nishcodezz AI Learning Playlist — <https://openup.to/islryggl>

SYSTEM DESIGN

- System Design Primer — <https://github.com/donnemartin/system-design-primer>

MOCK INTERVIEWS

- Pramp — <https://www.pramp.com/>
- Google Interview Warmup — <https://grow.google/certificates/interview-warmup/>
- LeetCode Interview — <https://leetcode.com/interview/>
- Interviewing.io — <https://interviewing.io/>

SALARY

- Levels.fyi — <https://www.levels.fyi/>
 - AmbitionBox — <https://www.ambitionbox.com/>
 - Glassdoor — <https://www.glassdoor.com/>
-

ONE LAST THING

You don't need to know everything before you start applying.

A better loop is:

LEARN → BUILD → APPLY → INTERVIEW → FIND WEAKNESS → IMPROVE → REPEAT

Don't spend six months “getting ready.”

Start interviewing before you feel 100% ready.

Because interview preparation becomes much easier when you have real interview feedback.

— Nishcodezz